

# Subrotinas e macros



**UNIFEI**

Universidade Federal de Itajubá

IMC – Instituto de Matemática e Computação

**Prof. Carlos Minoru Tamaki**

# Subrotinas

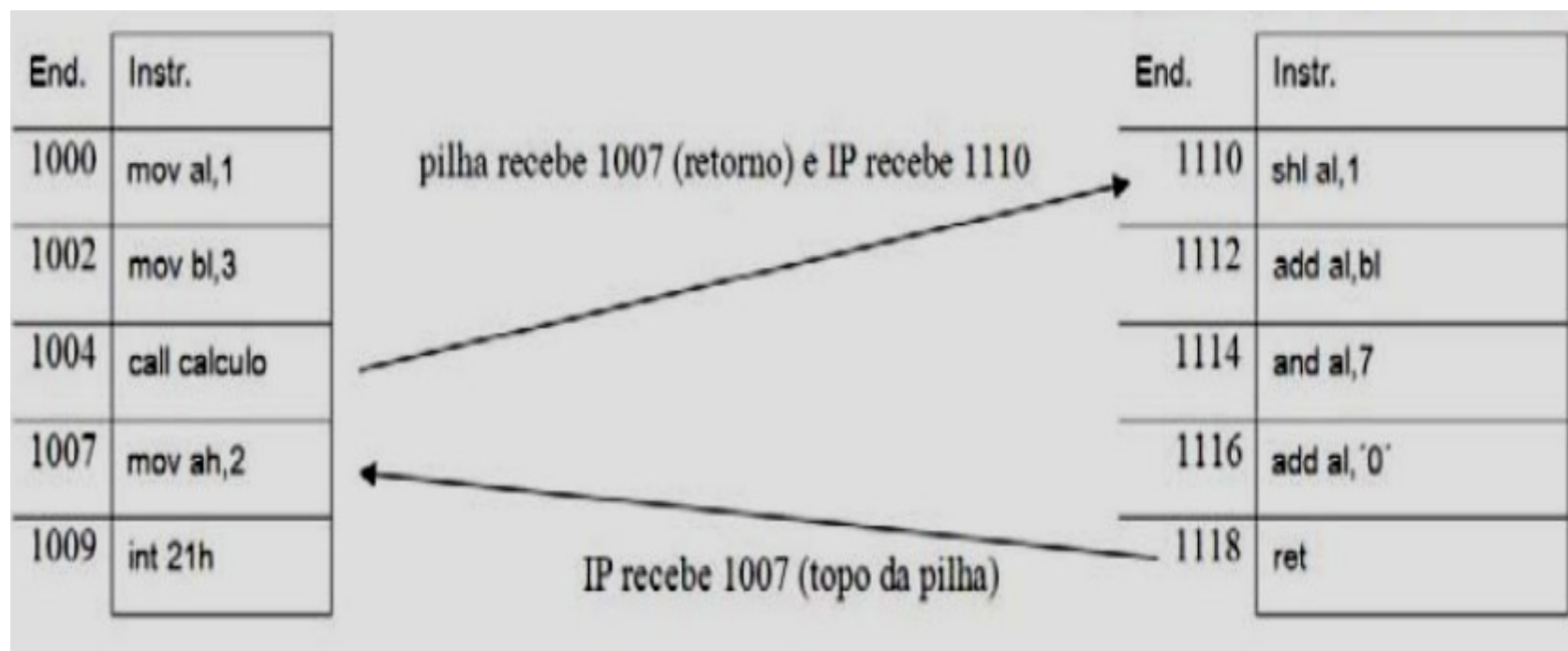
- Uma sub-rotina (função, procedimento ou mesmo subprograma) consiste em uma porção de código que resolve um problema muito específico, parte de um problema maior (a aplicação final).

- Algumas das vantagens na utilização de sub-rotinas durante a programação são:
  - a redução de código duplicado num programa;
  - a possibilidade de reutilizar o mesmo código sem grandes alterações em outros programas;
  - a decomposição de problemas grandes em pequenas partes;
  - melhorar a interpretação visual de um programa e
  - esconder ou regular uma parte de um programa, mantendo o restante código alheio às questões internas resolvidas dentro dessa função.

- Instrução CALL: chama uma sub-rotina, alterando o fluxo normal de execução.
  - Endereço de retorno é colocado na pilha pela instrução.
    - Quando uma instrução CALL é executada, o conteúdo de PC é armazenado na pilha (empilhado).
  - Sintaxe: CALL Proc
    - Proc: nome do procedimento (sub-rotina) a ser executado.

- Instrução RET: encerra uma sub-rotina, retomando a execução do programa chamador da sub-rotina.
  - Transfere o fluxo de processamento para a instrução seguinte à chamada da sub-rotina.
    - Desempilha o endereço armazenado na pilha e o coloca no registrador PC.
  - Sintaxe: RET

# Subrotinas



# Exemplo de Subrotina

```
section .text
    global _start ;deve ser
declarado para usar o gcc
_start: ;chamada para a
entrada do linker
    mov ecx, '4'
    sub ecx, '0'
    mov edx, '5'
    sub edx, '0'
    call sum ;chama o
procedimento sum
    mov [res], eax
    mov ecx, msg
    mov edx, len
    mov ebx, 1 ;file descriptor
(stdout)
    mov eax, 4 ;system call
number (sys_write)
    int 0x80 ;call kernel
    mov ecx, res
    mov edx, 1
```

```
    mov ebx, 1 ;file descriptor
(stdout)
    mov eax, 4 ;system call
number (sys_write)
    int 0x80 ;call kernel
    mov eax, 1 ;system call
number (sys_exit)
    int 0x80 ;call
kernel
```

## sum:

```
    mov eax, ecx
    add eax, edx
    add eax, '0'
    ret
```

```
section .data
msg db "The sum is:", 0xA, 0xD
len equ $- msg
```

```
section .bss
res resb 1
```

# Exemplo de Subrotina no Sasm

```
%include "io.inc"
```

```
section .text
```

```
global CMAIN
```

```
CMAIN:
```

```
    mov ecx, '4'
```

```
    sub ecx, '0'
```

```
    mov edx, '5'
```

```
    sub edx, '0'
```

```
    call sum ;chama o
```

```
procedimento sum
```

```
    mov [res], eax
```

```
    PRINT_STRING msg
```

```
    PRINT_CHAR [res]
```

```
    xor eax, eax
```

```
    ret
```

```
sum:
```

```
    mov eax, ecx
```

```
    add eax, edx
```

```
    add eax, '0'
```

```
    ret
```

```
section .data
```

```
msg db "A soma é: ", 0
```

```
section .bss
```

```
res resb 1
```

```
section .note.GNU-stack
```



# Macros

- Necessidade de repetir sequências de instruções
- Transformar a sequência em procedimento
  - Necessidade de uma instrução de chamada e uma de retorno
  - Lentidão devido sobrecarga de chamada de procedimento

# Definição, chamada e expansão de macro

; Uma macro com dois parametros  
; Implementa o write da chamada  
do sistema

```
%macro write_string      2
    mov    eax, 4
    mov    ebx, 1
    mov    ecx, %1
    mov    edx, %2
    int    80h
%endmacro
```

```
section      .text
    global _start      ;deve ser declarado
para usar gcc
_start:      ;ponto de entrada
do linker
    write_string msg1, len1
    write_string msg2, len2
    write_string msg3, len3
    mov     eax, 1      ;system call number
(sys_exit)
    int     0x80        ;call kernel
```

```
section      .data
msg1          db  'Hello, programadores!',
0xA, 0xD
len1 equ $ - msg1
msg2          db  'Benvindo ao mundo
do ', 0xA, 0xD
len2 equ $ - msg2
msg3          db  'Programador assembly
Linux! '
len3 equ $ - msg3
```

# Definição, chamada e expansão de macro na tradução

; Uma macro com dois parametros  
; Implementa o write da chamada  
do sistema

A linha:

`write_string msg1, len1`

`%macro write_string            2`

```
mov    eax, 4
mov    ebx, 1
mov    ecx, %1
mov    edx, %2
int    80h
```

`%endmacro`

Será substituída por:

```
mov    eax, 4
mov    ebx, 1
mov    ecx, msg1 ; msg1 é o
primeiro parâmetro %1
mov    edx, len1 ; len1 é o
segundo parâmetro %2
int    80h
```

E as demais linhas também.

O valor 2 em frente ao nome da macro significa que a macro possui 2 parâmetros que irá substituir nas posições %1 e %2, na sequência em que foram escritas.  
Caso seja omitido a macro não possui parâmetros.

# Exemplo de macro no Sasm

```
%include "io64.inc"
```

```
%macro change 2
```

```
; realiza a troca de dados entre  
dois registradores
```

```
    push %1
```

```
    mov %1, %2
```

```
    pop %2
```

```
%endmacro
```

```
section .text
```

```
global main
```

```
main:
```

```
    mov rax, 12345
```

```
    mov rdx, 0xbad
```

```
    PRINT_UDEC 8, rax
```

```
    NEWLINE
```

```
    PRINT_HEX 8, rdx
```

```
    NEWLINE
```

```
    change rax, rdx
```

```
    PRINT_UDEC 8, rax
```

```
    NEWLINE
```

```
    PRINT_HEX 8, rdx
```

```
    NEWLINE
```

```
    xor rax, rax
```

```
    ret
```

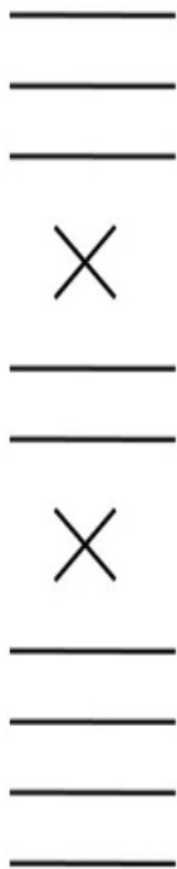
```
section .note.GNU-stack
```

# Comparação entre chamadas de macro e chamadas de procedimentos

| Item                                                                                           | Chamada de macro         | Chamada de procedimento      |
|------------------------------------------------------------------------------------------------|--------------------------|------------------------------|
| Quando a chamada é feita?                                                                      | Durante montagem         | Durante execução do programa |
| O corpo é inserido no programa-objeto em todos os lugares em que a chamada é feita?            | Sim                      | Não                          |
| Uma instrução de chamada de procedimento é inserida no programa-objeto e executada mais tarde? | Não                      | Sim                          |
| Deve ser usada uma instrução de retorno após a conclusão da chamada?                           | Não                      | Sim                          |
| Quantas cópias do corpo aparecem no programa-objeto?                                           | Uma por chamada de macro | Uma                          |

# Definição, chamada e expansão de macro na tradução

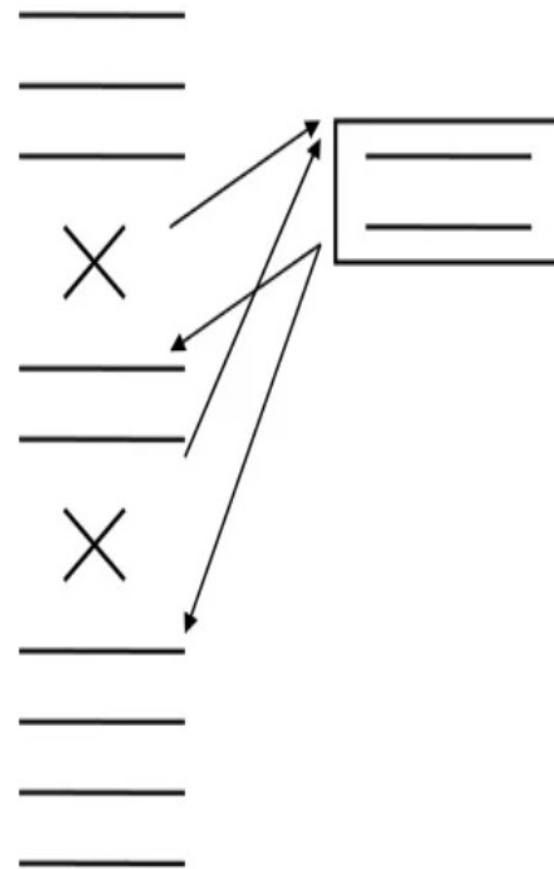
Programa



Macro



Subrotina



# Bibliografia

- Organização Estruturada de Computadores  
– Andrew S. Tanenbaum, Tood Austin – 6ª  
Edição 2013 Prentice Hall
- Vários sites da internet